
AI Safety Mini-Project 3

Inspecting Pretraining Data, Probing Harmfulness, and Steering Activations

Snehil Seenu¹ Maximilian Ohl¹ Lukáš Hypša¹

Part 1: Understanding and Filtering Pretraining Data

Task 1.1: Data Analysis

We sampled 30 documents from each of FineWeb, FinePDFs, C4, and SlimPajama-6B (substituted for RedPajama-V2; see Appendix) and annotated each one on topic, quality (1–10), cleaning correctness, and whether it contained private or harmful content.

Quality is similar in the middle but diverges at the tails. Mean quality landed in a narrow band, C4 5.47, FinePDFs 5.70, FineWeb 5.93, SlimPajama 5.87, with the median at 6 everywhere. The real difference shows up at the bottom: C4 had a 1, a 2, a 3, and a 4, while FineWeb’s lowest score was a single 2. The newer pipelines aren’t producing better best-case documents; they’re producing fewer catastrophic ones.

Each source has a recognisable shape. C4 was dominated by blog posts (11) and product pages (9). FineWeb was the most editorial: 23 of 30 documents were news (11) or blog posts (12). SlimPajama was a similar blog/news/product mix with more variety. FinePDFs was visibly different from the other three, mostly academic papers (9) and government documents (6), with structured artefacts (forms, financial documents, sports tables) that simply don’t appear when you scrape HTML.

Cleaning failures, not bad writing, drove most low scores. We marked 15/120 documents as poorly cleaned. The failure modes were source-specific: C4 leaked navigation menus and SEO spam (#71, scored 1, was repeated boilerplate phrases); FinePDFs produced table-heavy or garbled PDF extractions (#33, #53, #65); FineWeb and SlimPajama mostly leaked stray UI elements like shopping carts and filmography list fragments. The genuinely badly-written content was rare; *pipeline residue was the dominant low-quality signal*.

Private information was rare and entirely in FinePDFs. We flagged 3/120 documents as private, all in FinePDFs: a faculty seniority list with hire dates (#17), a government candidate selection list (#18), and a job-posting PDF with a visible recruiter email (#42). The web-scrape sources contributed zero. We flagged zero documents as harmful;

120 documents is a small slice of corpora that are billions of documents each, so this tells us the obvious harms are at most rare, not that they’re absent.

What surprised us. Two things, mainly. First, FinePDFs looked like a different category of training data: structured, academic, and the *only* source where we found PII. PDFs as a medium just carry different kinds of documents (faculty rosters, selection lists, HR materials) that web crawlers wouldn’t reach. Second, the C4-vs-FineWeb improvement was smaller than expected in absolute quality but visible at the failure tail. “Better cleaning” here means fewer catastrophic documents, not better average documents, a distinction worth noting when picking pipelines.

Part 1.2: Comparing our annotations to OpenAI’s privacy filter

We ran `openai/privacy-filter` on the same 120 documents and treated any flagged span as a document-level `private` prediction.

	Count
True positives (both flagged)	3
True negatives (neither flagged)	71
False positives (filter flagged, we did not)	46
False negatives (we flagged, filter did not)	0
Agreement rate	62%

Table 1. Filter vs annotator agreement, $n = 120$.

The filter caught everything we flagged (100% recall) but over-flagged heavily: 46 of 120 documents were false positives. Looking at the breakdown, `private_person` accounted for 40 of the 46 false-positive flags, journalist bylines, athletes in sports coverage, public figures in news articles, and authors in blog credits. The filter treats any named individual as PII; we don’t. That’s the entire story of the 62% agreement.

The per-source rates are interesting: the filter flagged 17/30 FineWeb documents (57%), more than any other source, despite FineWeb being the “cleanest” source by our quality scores. *Cleaner pipelines preserve more coherent text, which contains more named people, which the filter flags.* The filter is essentially counting how editorial a source is. One thing that stood out as a possible signal: the 3 docu-

ments we both flagged averaged **65.3 flagged entities each**, while the 46 false positives averaged **14.9**. That’s a $\sim 4\times$ density difference. Genuinely PII-bearing documents have lots of flags clustered together; over-flagged documents typically have a handful of incidental name mentions. A density threshold could plausibly recover most of the lost precision, though with $n = 3$ true positives we can’t fit one reliably, it’s a hypothesis worth testing on a larger annotated set.

Would we trust this filter to scrub a training corpus?

No, not as a binary document-level filter. It would discard 38% of documents that a human reader considers clean, mostly editorial content where the only “privacy concern” is that the article names the people it’s about. Used differently (redacting flagged spans rather than dropping whole documents, ideally with a density threshold), the same model might be useful. But that’s a different deployment than the binary classifier we evaluated.

2.1 Probing for Concepts

We probe the binary concept of harmfulness using a contrastive dataset with harmful prompts from AdvBench and harmless prompts from Alpaca. We use 150 examples per class, giving 300 prompts in total. The dataset is split into 240 training examples and 60 held out test examples. For each prompt, we apply the same chat template and cache residual stream activations from every layer. We then train one logistic regression probe per layer and evaluate probing accuracy on the test split.

At which layer is the concept most linearly decodable?

The concept is most linearly decodable at the last prompt token. The highest measured accuracy is obtained at layer 3, with an accuracy of 0.9833.

Is there a clear best layer or a broad plateau? The result is better interpreted as a broad plateau rather than one uniquely best layer. At the last token, probing accuracy becomes very high from layer 3 onward. After some variation in the middle layers, it remains especially stable from around layer 17 until the final layer.

What happens when probing other token positions?

The picture changes strongly across token positions. At the first token, accuracy stays around 0.45, which is close to chance level for a balanced binary classification task and therefore not informative. At the middle token, accuracy rises to roughly 0.75 to 0.92, showing that partial prompt context already contains information about harmfulness. At the last token, accuracy reaches roughly 0.95 to 0.98, giving the strongest and most stable signal.

How does the picture change across token positions?

Later token positions are more informative. The first token has almost no useful context. The middle token has access to part of the prompt and is therefore partially informative. The

last prompt token has access to the full preceding prompt and is therefore the best position for decoding harmfulness.

What does this suggest about where and when the concept is computed? The results suggest that harmfulness is represented through accumulated prompt context. The concept is not reliably available at isolated early tokens. It becomes increasingly decodable as more of the prompt has been processed, and it is most clearly linearly decodable at the end of the prompt.

2.2 Steering

For the second part with activation steering, we reused the same harmful-vs-harmless dataset with the same train/test split. While training split has been used to get the steering vector, test split was used later for the experiments. Based on the result from the previous part we computed the steering vector from last-token residual-stream activations.

The steering vector is being computed as a difference of means:

$$v_{\text{harm}} = \frac{\mu_{\text{harmful}} - \mu_{\text{harmless}}}{|\mu_{\text{harmful}} - \mu_{\text{harmless}}|}$$

The hidden state with index 20 has been used to compute the vector. Even though the index 3 already showed the best accuracy, its disadvantage might be it encode dataset-specific differences, rather than some deeper specific behaviour. That is the reason why we picked later state 20 with very similar accuracy.

During generation, we added the vector to the last-token residual stream:

$$h \leftarrow h + \alpha v$$

where α is the steering coefficient. Positive α should push the model towards the harmful trigger, while negative α should suppress that direction.

We evaluated 27 prompts. Refusal behavior was measured using a simple keyword detector, which was checking for such phrases as “I’m sorry,” “I cannot,” “illegal,” “harmful,” and similar. Due to the expensive computation cost, we evaluated only few alpha values.

The baseline model already refused all harmful prompts, giving a refusal rate of 1.0. Additive steering did not change this result: both positive steering with $\alpha = 4.0$ and negative steering with $\alpha = -4.0$ also produced refusal rates of 1.0. The small coefficient sweep was similarly flat, with both $\alpha = 0$ and $\alpha = 1$ giving refusal rates of 1.0.

At which layer and coefficient does steering work best; is there a sweet spot? In the current experiment, steering did not show a clear best layer or coefficient. The refusal rate was already 1.0 in the baseline condition, and it remained 1.0 for all tested steering coefficients, including $\alpha = 1$, $\alpha = 4$, and $\alpha = -4$. Therefore, there was no measurable improvement or suppression effect, and no clear sweet spot identified.

APPENDIX

Methodology: dataset substitution

We substituted SlimPajama-6B (DKYoon/SlimPajama-6B) for RedPajama-V2 because the official RedPajama-V2 distribution requires a deprecated dataset-loading script (`RedPajama-Data-V2.py`) that has been removed in `datasets` versions ≥ 3.0 . We tried several workarounds (downgrading the library, the parquet-only config with snapshot filters, and Cerebras’s SlimPajama-627B); the first two hit the same script error and the third was no longer available on the Hub. SlimPajama-6B is a community-uploaded parquet slice of Cerebras’s SlimPajama-627B, which is the cleaned and deduplicated version of RedPajama-1T sourced from the same corpora (Common Crawl, C4, GitHub, Books, ArXiv, Wikipedia, Stack Exchange). For this annotation task: “look at samples from a RedPajama-family dataset”, this substitution preserves the data family while remaining loadable on a modern environment.

Per-source statistics from Part 1.1

Source	N	Mean quality	Median quality	Cleaning bad	Mean chars
C4	30	5.47	6	4	2,089
FinePDFs	30	5.70	6	5	11,977
FineWeb	30	5.93	6	3	2,835
SlimPajama	30	5.87	6	3	2,658

Table 2. Per-source summary statistics from our 120-document annotation ($n = 30$ each). FinePDFs documents are roughly $4\text{--}6\times$ longer than the web-scrape sources, PDFs are inherently longer artefacts than typical web pages.

Topic distribution per source

Source	Topic counts (top categories)
C4	blog post (11), product page (9), nav menu (3), forum post (3)
FinePDFs	academic paper (9), government doc (6), product page (3), form (2), financial doc (2), sports data (2), event listing (2)
FineWeb	blog post (12), news article (11), product page (3)
SlimPajama	blog post (8), product page (8), news article (7), forum post (3)

Table 3. Topic distribution per source. FinePDFs is qualitatively different; the three web sources are blog/news/product-heavy with minor differences in mix.

Documents we flagged as private

ID	Source	Type	Why we flagged it
17	FinePDFs	academic	University seniority list with full faculty names and hire dates
18	FinePDFs	government	Government selection list with candidate names
42	FinePDFs	job posting	Recruiter email address visible in the document

Per-source filter rates (Part 1.2)

Source	Our flags / 30	Filter flags / 30	Filter rate
C4	0	5	17%
FinePDFs	3	14	47%
FineWeb	0	17	57%
SlimPajama	0	13	43%

Entity types in the 46 false-positive documents

Entity type	Occurrences
private_person	40
private_date	9
private_url	4
private_email	4
private_phone	3
private_address	2
account_number	1

Table 4. Entity types responsible for the 46 false-positive flags. A single document can contribute multiple types.

Flag-count density: TP vs FP

Group	Documents	Mean flagged_count	Median	Max
True positives	3	65.3	–	174
False positives	46	14.9	4	65
True negatives	71	0	0	0

Table 5. TP documents have $\sim 4\times$ the flag density of FP documents, suggesting flag-count could be used as a secondary discriminator. With $n = 3$ TP, however, the threshold cannot be fit reliably and the conclusion is suggestive rather than statistical.